

Introduction to Linked list

- * A linked list is a linear data structure in which element (called nodes) are connected using pointers.
- * Unlike arrays, linked list are not stored in contiguous memory location.
- * Unlike of array, in case of linked list we can keep adding and removing elements without capacity constraint.

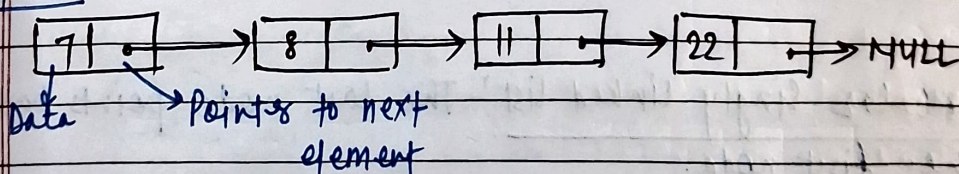
Drawbacks:

- * Extra memory space for pointer is required
- * Random access is not allowed as elements are not stored in contiguous memory locations.

7	8	11	22
---	---	----	----

 → Array (Stored in contiguous Memory).

Linked list



(Stores actual value) (Stores address of next node)

Head :- Points to the first node.

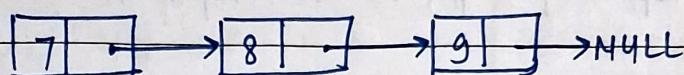
NULL :- Indicate the end of the list.

Features of linked list

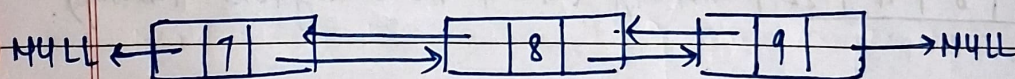
- * Dynamic size (can grow or shrink during runtime).
- * Efficient insertion and deletion.
- * Does not require contiguous memory.
- * Sequential access only.

Types of linked list

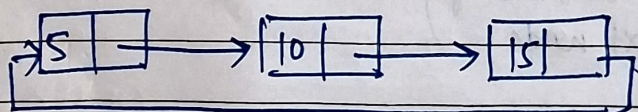
(i) Singly linked list:- Each node has one pointer pointing to the next node.



(ii) Doubly linked list:- Each node has two pointers that denote to the previous pointer and the next pointer.



(iii) Circular singly linked list:- The last node points back to the first node.



(iv) Circular doubly linked list:- Both previous and next pointers form a circle.

Implementation:- linked list can be implemented using a structure in C language.

Page No. _____

Date _____

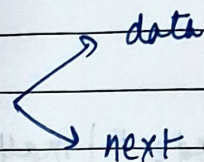
struct Node {

int data

struct Node *next; $\Rightarrow$ self referencing structure.
};

Creation of linked list

* It means creating nodes dynamically and connecting them using pointers

* Each node contains 

Traversal:-

It means visiting each node of the linked list one by one.

temp = head;

```
while (temp != NULL) {  
    print ("%d", temp->data);  
    temp = temp->next;  
}
```


Program:- Creation + Traversal of linked list



Page No. _____

Date _____

```
#include <stdio.h>
```

```
#include <stdlib.h> // used when malloc is used.
```

```
struct Node{  
    int data;  
    struct Node *next;  
};
```

```
int main()
```

```
    struct Node *head, *second, *third;
```

```
// Create nodes.
```

```
    head = (struct Node *) malloc (size of (struct Node));
```

```
    second = (struct Node *) malloc (size of (struct Node));
```

```
    third = (struct Node *) malloc (size of (struct Node));
```

```
// Store data and connect nodes
```

for Traversal

```
head → data = 10;
```

```
head → next = second;
```

```
second → data = 20;
```

```
second → next = third;
```

```
third → data = 30;
```

```
third → next = NULL;
```

Time Complexity :- $O(n)$

no. of
nodes

Space Complexity :- $O(1)$.

```
// Traversal.
```

```
struct Node *temp = head;
```

```
while (temp != NULL) {
```

```
    printf("%d", temp → data);
```

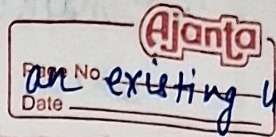
```
    temp = temp → next;
```

```
}
```

```
return 0;
```


Insertion in linked list

It means adding a new node to an existing linked list.



Types of Insertion

(i) Insertion at Beginning: It means adding a new node before the current first node.

Time Complexity: $O(1)$ $\rightarrow$ becoz we don't need to traverse the list.

Space Complexity: $O(1)$ $\rightarrow$ for the newly created node.

Steps:

- * Create a new node
- * store data in it
- * Make `newNode  $\rightarrow$  data = head;`
- * Make `head = newNode;`

Program: Insertion at Beginning.

```
#include <stdio.h>
#include <stdlib.h>
```

```
struct Node{
    int data;
    struct Node *next;
};
```


int main() {

struct Node * head = NULL;

struct Node * new Node;

head = (struct Node *) malloc(sizeof(struct Node));

head → data = 10;

head → next = NULL;

new Node = (struct Node *) malloc(sizeof(struct Node));

new Node → data = 5;

new Node → next = head;

head = new Node;

struct Node * temp = head;

while (temp != NULL) {

print("%d", temp → data);

temp = temp → next;

}

return 0;

}

(ii) Insertion at end:- Adding the new node at the end.

Time Complexity = $O(n)$

* Create a new node

* Traverse to the last node

* Make the last node point to the new node

* Set new Node → next = NULL

Program:- Insertion at End:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct Node {  
    int data;  
    struct Node *next;  
};
```

```
struct Node *head = NULL;
```

```
void InsertEnd(int value) {  
    struct Node *newNode;  
    struct Node *temp;
```

```
    newNode = (struct Node *) malloc ( sizeof ( struct Node ));  
    newNode->data = value;  
    newNode->next = NULL;
```

```
// If list is empty.
```

```
    if (head == NULL) {  
        head = newNode;  
        return;
```

```
    }
```

```
// Traverse to the last node.
```

```
    temp = head;  
    while (temp->next != NULL) {  
        temp = temp->next;  
    }
```

```
    temp->next = newNode;
```

```
}
```



```
void display() {
```

```
    struct Node * temp = head;
```

```
    while (temp != NULL) {
```

```
        printf("%d", temp->data);
```

```
        temp = temp->next;
```

```
    }
```

```
    printf("NULL\n");
```

```
}
```

```
int main() {
```

```
    InsertEnd(10);
```

```
    InsertEnd(20);
```

```
    InsertEnd(30);
```

```
    display();
```

```
    return 0;
```

```
}
```

(iii) Insertion at a specific position (or After a given Node)

- * Create a new node.

- * Traverse to the node after which insertion is required.

- * Set the ~~new~~ new node's next to current node's next.

- * Set the current node's next to the new Node.

Time complexity :- $O(n)$

Program: Insertion at specific position

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct Node {  
    int data;  
    struct Node *next;  
};
```

```
int main() {
```

```
    struct Node *head, *newNode, *temp;  
    int position = 3;
```

```
    head = (struct Node*) malloc (sizeof (struct Node));  
    head->data = 10;
```

```
    head->next = (struct Node*) malloc (sizeof (struct Node));  
    head->next->data = 20;
```

```
    head->next->next = (struct Node*) malloc (sizeof (struct Node));  
    head->next->next->data = 40;
```

```
    head->next->next->next = NULL;
```

/// Create a new node

```
newNode = (struct Node*) malloc (sizeof (struct Node));  
newNode->data = 30;
```

~~temp = newNode;~~


```
temp = head;
for (int i = 0; i < position - 1; i++) {
    temp = temp -> next;
}
```

Page No. _____
Date _____

Ajanta

// Insert Node.

```
newNode -> next = temp -> next;
temp -> next = newNode;
}
```

// Display list.

```
temp = head;
while (temp != NULL) {
    printf ("%d", temp -> data);
    temp = temp -> next;
}
```

```
return 0;
}
```

Deletion in singly linked list

Deletion in a linked list means removing a node from the linked list and adjusting the links so that the remaining nodes stay connected.

Types of Deletion

- (i) Deletion from the beginning.
- (ii) Deletion from the end.
- (iii) Deletion from a specific position.
- (iv) Deletion of a node with a specific value.

Deletion from Beginning

Page No.

Date

Algorithm:- (i) Check whether the list is empty.
(ii) Store the first node in a temporary pointer.

(iii) Move head to the second node
(iv) Free the old first node.

* Time Complexity:- $O(1)$

Program:- ~~Deletion~~ Deletion from Beginning

```
#include <stdio.h>
#include <stdlib.h>
```

```
struct Node {
    int data;
    struct node *next;
};
```

```
void deleteBeginning (struct Node *head) {
    if (*head == NULL) {
        printf ("Empty list \n");
    }
```

```
    struct Node *temp = *head;
    *head = (*head) -> next;
    free (temp);
}
```

```
void display (struct Node *head) {
    while (head != NULL) {
        printf ("x.d: ", head->data);
        head = head->next;
    } -> print ("NULL \n");
}
```



```
int main() {
```

```
    struct Node *head = NULL;
```

```
    struct Node *n1 = malloc (size of (struct Node));
```

```
    "      " * n2 = "      " "      " "      " ;
```

```
    "      " * n3 = "      " "      " "      " ;
```

```
    n1->data = 10;
```

```
    n1->next = n2;
```

```
    n2->data = 20;
```

```
    n2->next = n3;
```

```
    n3->data = 30;
```

```
    n3->next = NULL;
```

```
    head = n1;
```

```
    delete Beginning(&head);
```

```
    printf (" After deletion: \n");
```

```
    display (head);
```

```
    return 0;
```

```
}
```


Deletion from the End

Algorithm:- (i) check if the list is empty.

- (ii) if there is only one last node, delete it and set head = NULL.
- (iii) otherwise, transverse to the select last node.
- (iv) store the last node in temp.
- (v) ~~delete the~~ set the second-last node's next to NULL.
- (vi) free the last node.

Time Complexity:- $O(n)$

Program:- deletion from the End.

```
void DeleteEnd (struct Node **head) {  
    if (*head = NULL) {  
        printf ("Empty list \n");  
        return;  
    }  
}
```

```
struct Node * temp = *head;
```

// only one head.

```
if (temp->next = NULL) {  
    *head = NULL;  
    free (temp);  
    return;  
}
```

// Move to second-last node.

```
while (temp->next->next != NULL) {  
    temp = temp->next;  
}
```

```
struct Node * last = temp->next;  
temp->next = NULL;
```

```
free (last);
```

```
printf ("Node deleted \n");  
}
```


Deletion from a specific position.

Algorithm:- (i) Check if the list is empty.

(ii) If position is 1, delete the first node.

(iii) Traverse to the node before the position.

(iv) Store the node to be deleted in temp.

(v) change the previous node's next

(vi) free temp.

Time Complexity:- $O(n)$

Program:- Deletion from a specific position

```
void deletePosition(struct Node **head, int position){  
    if (*head == NULL){  
        printf("Empty list\n");  
        return;  
    }  
}
```

```
if (position <= 0){  
    printf("Invalid posn\n");  
    return;  
}
```

// delete first node.

```
if (position == 1){  
    struct Node *temp = head;  
    *head = (*head) -> next;  
    free(temp);  
}
```

```
printf("Node deleted\n");
```


// Reach node before Required position

struct Node *temp = head;

```
for(int i=1; i < position-1 && temp != NULL; i++){  
    temp = temp->next;  
}
```

```
if (temp == NULL || temp->next == NULL) {  
    printf ("Position does not exists \n");  
    return;  
}
```

```
struct Node *deletenode = temp->next;  
temp->next = deletenode->next;  
free (deletenode);
```

```
printf ("Node deleted from position %d \n", position);  
}
```

NOTE:- Why free() is important

Why node is deleted its memory should be released using free.

If free() is not used, the node is removed from the list logically, but its allocated memory remains occupied.

This can cause memory leak.

Step-by-Step Dry Run

Head.

↓
0 → 20 → 30 → 40 → NULL

* Initialize temp

temp

↓
10 → 20 → 30 → 40 → NULL

* Position = 3

Initialize.

* Loop → $i = 1$ $1 < ((3-1)=2) \rightarrow \text{True}$.

temp = temp → next (temp now pts to 20)

10 → 20 → 30 → 40 → NULL
 ↑
 temp

Next $i = 2$ $(2 < 2) \rightarrow \text{False}$.

loops stop.

* del → temp → next (del points to 30)

10 → 20 → 30 → 40 → NULL
 ↑ ↑
 temp del.

* temp → next = del → next

(temp → next = 40)

10 → 20 → 40 → NULL
 ↓ ↓
 temp temp → next

* free the node :- free(del)

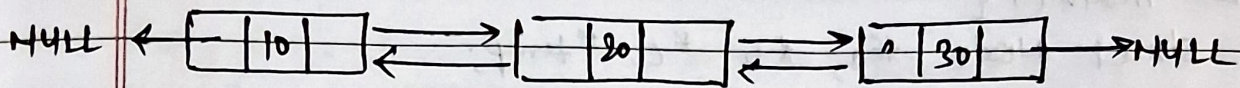
Doubly linked list

A doubly linked list is a linked list in which each node contains three parts:-

prev $\rightarrow$ points to the previous node

data $\rightarrow$ stores the value

next $\rightarrow$ points to the next node.



Creation of doubly linked list

Algorithm:- (i) Create a new node using malloc()

(ii) Store data in the node.

(iii) Set next and prev pointers

(iv) Connect the new node with previous node.

(v) Repeat until all nodes are created.

Traversal of doubly linked list

It means visiting every node.

Algorithm:- (i) Set temp = head

(ii) while temp $\neq$ NULL

 print temp $\rightarrow$ data

 Move temp to temp $\rightarrow$ next.

(iii) Stop:

Program:- Doubly linked list Creation + Traversal

```
#include <stdio.h>
#include <stdlib.h>
```

```
struct Node {
    int data;
    struct Node *prev, *next;
};
```

```
int main() {
    struct Node *a, *b, *c, *temp;
```

```
    a = malloc (sizeof (struct Node));
```

```
    b = " " " " ;
```

```
    c = " " " " ;
```

—————> a → data = 10;

```
    a → prev = NULL;
```

b → data = 20;

```
    a → next = b;
```

c → data = 30;

```
    b → prev = a;
```

```
    b → next = c;
```

```
    c → prev = b;
```

```
    c → next = NULL;
```

// Forward Traversal.

```
temp = a;
while (temp != NULL) {
```

```
    printf ("%d", temp → data);
    temp = temp → next;
}
```

// Backward Traversal

```
temp = c;
```

```
while (temp != NULL) {
```

```
    printf ("%d", temp → data);
    temp = temp → prev;
}
return 0;
}
```


Insertion in Doubly linked list

(i) At beginning:-

Time Complexity:- $O(1)$

Algorithm:- * Create a new node

* Set new node $\rightarrow$ prev = NULL

* Set new node $\rightarrow$ next = head

* If head is not NULL, set head $\rightarrow$ prev = new

* Set head = new.

Program:- Insertion in beginning (Doubly linked list)

void insertBeginning (struct Node **head, int value) {

struct Node *new = malloc (size of (struct Node));

new $\rightarrow$ data = value;

~~new $\rightarrow$ prev = NULL;~~ new $\rightarrow$ prev = NULL;

new $\rightarrow$ next = head;

$\rightarrow$ if (*head != NULL)

* head = new;

(*head) $\rightarrow$ prev = new;

(ii) Insertion at End:-

Algorithm:- * Create a new node

* Traverse to the last node

* Set the last node's next to new node.

* Set the new node's prev to last node,

* Set the new node's next = NULL.

Time Complexity:- $O(n)$

Program:- Insertion at End (Doubly linked list)

```
void insertEnd (struct Node ** head, int value) {  
    struct Node * new = malloc (sizeof (struct Node));  
  
    struct Node * temp = head;  
  
    new->data = value;  
    new->next = NULL;  
  
    while (temp->next != NULL)  
        temp = temp->next;  
  
    temp->next = new;  
    new->prev = temp;  
}
```

(ii) Insertion at specific position :-

Algorithm:- * Create a new node and store data

- * Traverse the node at pos-1.
- * Set new->next = temp->next.
- * Set new->prev = temp.
- * Set temp->next = new
- * If new->next != NULL, set new->next->prev = new.
- * Stop.

Time Complexity:- $O(n)$.

Program:- Insertion at specific position (Doubly linked list)

Page No.	
Date	

```
void insertPosition (struct Node *head, int value, int pos){  
    struct Node *temp = head;
```

```
    struct Node new = malloc (sizeof (struct Node));
```

```
    new->data = value;
```

```
    for (int i=0 ; i< pos-1 ; i++){  
        temp = temp->next;
```

```
        new->next = temp->next;  
        new->prev = temp;
```

```
        temp->next->prev = new;  
        temp->next = new;
```

```
    }
```

Deletion in Doubly linked list

(i) At beginning:-

Algorithm:- * store head in temp.

* Move head to the next node.

* Set next node's prev = NULL;

* free the old node.

Time Complexity :- $O(1)$

Program:- Deletion from Beginning (Doubly linked list)

Page No. _____

Date _____

```
void deleteBeginning (struct Node **head) {  
    struct Node **temp = *head;
```

```
    *head = (*head) → next;
```

```
    if (*head != NULL)
```

```
        (*head) → prev = NULL;
```

```
    free (temp);
```

```
}
```

(ii) Deletion from End:-

Time Complexity:- $O(n)$

Algorithm:-

- * Traverse to the last node
- * set the previous node's next = NULL
- * free the last node.

Program:- Deletion from End (Doubly linked list)

```
void deleteEnd (struct Node *head) {  
    struct Node *temp = head;
```

```
    while (temp → next != NULL)
```

```
        temp = temp → next;
```

```
    temp → prev → next = NULL;
```

```
    free (temp);
```

```
}
```


(iii) Deletion from specific position

Algorithm:- * Traverse to the node at given position.

* If it is the first node, update head.

* set $\text{temp} \rightarrow \text{prev} \rightarrow \text{next} = \text{temp} \rightarrow \text{next}$

* If $\text{temp} \rightarrow \text{next} \neq \text{NULL}$ set $\text{temp} \rightarrow \text{next} \rightarrow \text{prev} = \text{temp} \rightarrow \text{prev}$

* free the node temp.

* stop.

Time Complexity:- $O(n)$

Program:- Deletion from specific position (doubly linked list)

```
void deletePosition (struct Node *head, int pos) {  
    struct Node *temp = head;
```

```
    for (int i = 1; i < pos; i++) {  
        temp = temp -> next;
```

```
    temp -> prev -> next = temp -> next;
```

```
    if (temp -> next != NULL)  
        temp -> next -> prev = temp -> prev;
```

```
    free (temp);
```

```
}
```